

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет «Информатика и системы управления»
Кафедра «Системы обработки информации и управления»

Домашнее задание по курсу
Парадигмы и конструкции языков программирования
ГУИМЦ

Исполнитель: **Кузнецов Н. В.**
студент группы
ИУ5Ц-54Б

Преподаватель: **Гапанюк Ю. Е.**

Москва, МГТУ – 2025

Содержание

Цель домашнего задания	3
Задание	4
Часть 1: Установка и настройка	5
Установка Rust на Windows.....	5
Настройка VS Code.....	5
Часть 2: Реализация проекта - "Система управления задачами"	6
Описание проекта	6
Структура проекта	6
Полный код программы.....	6
Компиляция и запуск	12
Часть 3: Экранные формы (Примеры работы программы).....	14
Запуск программы.....	14
Добавление задачи	14
Просмотр задач	14
Сохранение в файл.....	14
Содержимое файла tasks.json.....	15
Часть 4: Выводы	16
Преимущества Rust, обнаруженные при изучении:	16
Сложности при изучении:	16
Заключение:	16

Цель домашнего задания

изучение ранее неизвестного языка программирования

Задание

1. Выберите язык программирования (который Вы ранее не изучали) и (1) напишите по нему реферат с примерами кода или (2) реализуйте на нем небольшой проект (с детальным текстовым описанием).
2. Реферат (проект) может быть посвящен отдельному аспекту (аспектам) языка или содержать решение какой-либо задачи на этом языке.
3. Необходимо установить на свой компьютер компилятор (интерпретатор, транспилятор) этого языка и произвольную среду разработки.
4. В случае написания реферата необходимо разработать и откомпилировать примеры кода (или модифицировать стандартные примеры).
5. В случае создания проекта необходимо детально комментировать код.
6. При написании реферата (создании проекта) необходимо изучить и корректно использовать особенности парадигмы языка и основных конструкций данного языка.
7. Приветствуется написание черновика статьи по результатам выполнения ДЗ. Черновик статьи может быть подготовлен группой студентов, которые исследовали один и тот же аспект в нескольких языках или решили одинаковую задачу на нескольких языках.

Часть 1: Установка и настройка

Установка Rust на Windows

1. Скачиваем установщик с официального сайта: <https://rust-lang.org/ru/learn/get-started/>
2. Запускаем rustup-init.exe
3. В командной строке следуем инструкциям установщика
4. Проверяем установку:

```
C:\Users\user>rustc --version
rustc 1.91.1 (ed61e7d7e 2025-11-07)

C:\Users\user>cargo --version
cargo 1.91.1 (ea2d97820 2025-10-10)
```

Настройка VS Code

Устанавливаем расширения:

- Rust-analyzer
- Dependi
- Even Better TOML
- Error Lens

Создаем тестовый проект:

```
C:\Users\user\vs my code\PCPL-labs-2025\dz>cargo new rust_project
  Creating binary (application) `rust_project` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
```

Часть 2: Реализация проекта - "Система управления задачами"

Описание проекта

Проект представляет собой консольное приложение для управления задачами с возможностями:

1. Добавление задач
2. Просмотр всех задач
3. Отметка задач как выполненных
4. Удаление задач
5. Сохранение задач в файл
6. Загрузка задач из файла

Структура проекта

```
rust_project/  
----Cargo.toml  
----src/  
-----main.rs  
----tasks.json
```

Полный код программы

Cargo.toml:

```
[package]  
name = "rust_project"  
version = "0.1.0"  
edition = "2024"  
  
[dependencies]  
serde = { version = "1.0", features = ["derive"] }  
serde_json = "1.0"  
chrono = "0.4"
```

src/main.rs:

```
// Импорт необходимых библиотек
use serde::{Deserialize, Serialize};
use serde_json;
use std::fs;
use std::io;
use chrono::prelude::*;

// Структура задачи
#[derive(Debug, Serialize, Deserialize, Clone)]
struct Task {
    id: u32,
    title: String,
    description: String,
    completed: bool,
    created_at: String,
    completed_at: Option<String>,
}

// Структура менеджера задач
struct TaskManager {
    tasks: Vec<Task>,
    next_id: u32,
}

impl TaskManager {
    // Создание нового менеджера задач
    fn new() -> Self {
        TaskManager {
            tasks: Vec::new(),
            next_id: 1,
        }
    }

    // Добавление новой задачи
    fn add_task(&mut self, title: String, description: String) {
        let task = Task {
            id: self.next_id,
            title,
            description,
            completed: false,
            created_at: Local::now().to_rfc3339(),
            completed_at: None,
        };

        self.tasks.push(task);
        self.next_id += 1;
        println!("Задача успешно добавлена!");
    }
}
```

```

// Просмотр всех задач
fn view_tasks(&self) {
    if self.tasks.is_empty() {
        println!("Список задач пуст.");
        return;
    }

    println!("\n=== СПИСОК ЗАДАЧ ===");
    for task in &self.tasks {
        let status = if task.completed { "✓" } else { "X" };
        println!("ID: {}", task.id);
        println!("Заголовок: {}", task.title);
        println!("Описание: {}", task.description);
        println!("Статус: {}", status);
        println!("Создана: {}", task.created_at);
        if let Some(completed_at) = &task.completed_at {
            println!("Завершена: {}", completed_at);
        }
        println!("---");
    }
}

// Отметка задачи как выполненной
fn complete_task(&mut self, id: u32) -> Result<(), String> {
    for task in &mut self.tasks {
        if task.id == id {
            if !task.completed {
                task.completed = true;
                task.completed_at = Some(Local::now().to_rfc3339());
                return Ok(());
            } else {
                return Err("Задача уже выполнена".to_string());
            }
        }
    }
    Err("Задача с указанным ID не найдена".to_string())
}

// Удаление задачи
fn delete_task(&mut self, id: u32) -> Result<(), String> {
    let initial_len = self.tasks.len();
    self.tasks.retain(|task| task.id != id);

    if self.tasks.len() < initial_len {
        Ok(())
    } else {
        Err("Задача с указанным ID не найдена".to_string())
    }
}

// Сохранение задач в файл

```



```

fn save_to_file(&self, filename: &str) -> io::Result<()> {
    let json = serde_json::to_string_pretty(&self.tasks)?;
    fs::write(filename, json)?;
    println!("Задачи сохранены в файл '{}'", filename);
    Ok(())
}

// Загрузка задач из файла
fn load_from_file(&mut self, filename: &str) -> io::Result<()> {
    let data = fs::read_to_string(filename)?;
    let tasks: Vec<Task> = serde_json::from_str(&data)?;

    // Находим максимальный ID для продолжения нумерации
    self.next_id = tasks.iter().map(|t| t.id).max().unwrap_or(0) + 1;
    self.tasks = tasks;
    println!("Задачи загружены из файла '{}'", filename);
    Ok(())
}
}

// Главная функция
fn main() {
    println!("=== СИСТЕМА УПРАВЛЕНИЯ ЗАДАЧАМИ ===");

    let mut task_manager = TaskManager::new();
    let filename = "tasks.json";

    // Попытка загрузить задачи из файла при запуске
    if let Err(e) = task_manager.load_from_file(filename) {
        println!("Не удалось загрузить задачи: {}. Будет создан новый список.",
e);
    }

    loop {
        println!("\nМеню:");
        println!("1. Добавить задачу");
        println!("2. Просмотреть задачи");
        println!("3. Завершить задачу");
        println!("4. Удалить задачу");
        println!("5. Сохранить задачи");
        println!("6. Выход");
        println!("Выберите действие: ");

        let mut choice = String::new();
        io::stdin().read_line(&mut choice).expect("Ошибка чтения ввода");

        match choice.trim() {
            "1" => {
                println!("Введите заголовок задачи: ");
                let mut title = String::new();
                io::stdin().read_line(&mut title).expect("Ошибка чтения ввода");
            }
        }
    }
}

```

```

println!("Введите описание задачи: ");
let mut description = String::new();
io::stdin().read_line(&mut description).expect("Ошибка чтения
ввода");

task_manager.add_task(
    title.trim().to_string(),
    description.trim().to_string()
);
}

"2" => {
    task_manager.view_tasks();
}

"3" => {
    println!("Введите ID задачи для завершения: ");
    let mut id_input = String::new();
    io::stdin().read_line(&mut id_input).expect("Ошибка чтения
ввода");

    match id_input.trim().parse::<u32>() {
        Ok(id) => {
            match task_manager.complete_task(id) {
                Ok(_) => println!("Задача {} отмечена как
выполненная!", id),
                Err(e) => println!("Ошибка: {}", e),
            }
        }
        Err(_) => println!("Неверный формат ID"),
    }
}

"4" => {
    println!("Введите ID задачи для удаления: ");
    let mut id_input = String::new();
    io::stdin().read_line(&mut id_input).expect("Ошибка чтения
ввода");

    match id_input.trim().parse::<u32>() {
        Ok(id) => {
            match task_manager.delete_task(id) {
                Ok(_) => println!("Задача {} удалена!", id),
                Err(e) => println!("Ошибка: {}", e),
            }
        }
        Err(_) => println!("Неверный формат ID"),
    }
}
}

```

```

        "5" => {
            if let Err(e) = task_manager.save_to_file(filename) {
                println!("Ошибка при сохранении: {}", e);
            }
        }

        "6" => {
            // Автосохранение при выходе
            if let Err(e) = task_manager.save_to_file(filename) {
                println!("Ошибка при автосохранении: {}", e);
            }
            println!("До свидания!");
            break;
        }

        _ => {
            println!("Неверный выбор. Попробуйте снова.");
        }
    }
}

// Тесты
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_add_task() {
        let mut manager = TaskManager::new();
        manager.add_task("Тест".to_string(), "Описание теста".to_string());

        assert_eq!(manager.tasks.len(), 1);
        assert_eq!(manager.tasks[0].title, "Тест");
        assert_eq!(manager.next_id, 2);
    }

    #[test]
    fn test_complete_task() {
        let mut manager = TaskManager::new();
        manager.add_task("Тест".to_string(), "Описание".to_string());

        let result = manager.complete_task(1);
        assert!(result.is_ok());
        assert!(manager.tasks[0].completed);
    }

    #[test]
    fn test_delete_task() {
        let mut manager = TaskManager::new();
        manager.add_task("Тест".to_string(), "Описание".to_string());
    }
}

```

```
    let result = manager.delete_task(1);  
    assert!(result.is_ok());  
    assert!(manager.tasks.is_empty());  
  }  
}
```

Компиляция и запуск

Сборка проекта:

```
C:\Users\user\vs my code\PCPL-labs-2025\dz\rust_project>cargo build  
Compiling proc-macro2 v1.0.103  
Compiling quote v1.0.42  
Compiling serde_core v1.0.228  
Compiling serde_json v1.0.145  
Compiling serde v1.0.228  
Compiling num-traits v0.2.19  
Compiling chrono v0.4.42  
Compiling syn v2.0.111  
Compiling serde_derive v1.0.228  
Compiling rust_project v0.1.0 (C:\Users\user\vs my code\PCPL-labs-  
2025\dz\rust_project)  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 6.06s
```

Запуск проекта:

```
C:\Users\user\vs my code\PCPL-labs-2025\dz\rust_project>cargo run  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.02s  
Running `target\debug\rust_project.exe`  
=== СИСТЕМА УПРАВЛЕНИЯ ЗАДАЧАМИ ===  
Не удалось загрузить задачи: Не удастся найти указанный файл. (os error 2). Будет  
создан новый список.
```

Меню:

1. Добавить задачу
2. Просмотреть задачи
3. Завершить задачу
4. Удалить задачу
5. Сохранить задачи
6. Выход

Выберите действие:

Запуск тестов:

```
C:\Users\user\vs my code\PCPL-labs-2025\dz\rust_project>cargo test
  Compiling rust_project v0.1.0 (C:\Users\user\vs my code\PCPL-labs-2025\dz\rust_project)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.29s
    Running unittests src\main.rs (target\debug\deps\rust_project-10ff74f5ad57c4f9.exe)

running 3 tests
test tests::test_add_task ... ok
test tests::test_delete_task ... ok
test tests::test_complete_task ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

Часть 3: Экранные формы (Примеры работы программы)

Запуск программы

```
=== СИСТЕМА УПРАВЛЕНИЯ ЗАДАЧАМИ ===  
Задачи загружены из файла 'tasks.json'  
  
Меню:  
1. Добавить задачу  
2. Просмотреть задачи  
3. Завершить задачу  
4. Удалить задачу  
5. Сохранить задачи  
6. Выход  
Выберите действие:
```

Добавление задачи

```
Выберите действие:  
1  
Введите заголовок задачи:  
Изучить Rust  
Введите описание задачи:  
Прочитать документацию и написать проект  
Задача успешно добавлена!
```

Просмотр задач

```
Выберите действие:  
2  
  
=== СПИСОК ЗАДАЧ ===  
ID: 1  
Заголовок: Изучить Rust  
Описание: Прочитать документацию и написать проект  
Статус: X  
Создана: 2025-12-11T02:22:55.346267300+03:00  
---
```

Сохранение в файл

```
Выберите действие:  
5  
Задачи сохранены в файл 'tasks.json'
```

Содержимое файла tasks.json

```
[
  {
    "id": 1,
    "title": "Изучить Rust",
    "description": "Прочитать документацию и написать проект",
    "completed": false,
    "created_at": "2025-12-11T02:22:55.346267300+03:00",
    "completed_at": null
  }
]
```

Часть 4: Выводы

Преимущества Rust, обнаруженные при изучении:

1. Безопасность памяти без сборщика мусора - система владения и заимствования предотвращает распространенные ошибки.
2. Высокая производительность - сравнима с C/C++, но с большей безопасностью.
3. Отличная система типов - строгая типизация с выводом типов.
4. Эффективная обработка ошибок - явное использование Result и Option.
5. Мощные средства метапрограммирования - атрибуты, макросы.
6. Отличная документация - Cargo doc, rustdoc.
7. Современный инструментарий - Cargo как единый инструмент для сборки, зависимостей, тестирования.

Сложности при изучении:

1. Кривая обучения - система владения требует переосмысления подхода к управлению памятью.
2. Время компиляции - может быть длительным для больших проектов.
3. Строгость компилятора - иногда требуется много времени на исправление ошибок заимствования.

Заключение:

Rust представляет собой современный системный язык программирования, сочетающий производительность C++ с безопасностью высокоуровневых языков. Его изучение требует времени и усилий, но результаты оправдывают вложения - создание надежного, безопасного и эффективного программного обеспечения. Язык особенно подходит для системного программирования, встраиваемых систем, веб-серверов и других задач, где важны безопасность и производительность.